

SSD_FLASH&DDR usage reference

1. Flash partition view and configuration

1.1. How to view the partition configuration

a. In the compilation stage, after compiling the project, the corresponding partition allocation log will be printed on the serial port:

```
BOOT: 0x140000(CIS),0x200000@0x140000(IPL0)2,0x20000(IPL1)2,0x20000(IPL_CUST0)2,0x20000(IPL_CUST1)2,0x40000(UBOOT0)4,0x40
SYS: 0x20000(KEY_CUST),0x60000(LOGO),0x500000(KERNEL),0x500000(RECOVERY),-(UBI)
FLASH HAS USED 0x8000000KB
ChkSum      : 1126
SpareByteCnt : 64
PageByteCnt  : 2048
BlkPageCnt   : 64
BlkCnt       : 1024
PartCnt      : 14
UnitByteCnt  : 8
Checksum ok!!
IDX:      StartBlk:      BlkCnt:      BackupBlkCnt:      PartType:
0:      0,(0000000000)  10,(0X00140000)  0,(0000000000)  0X0020,(CIS)
1:      10,(0X00140000)  1,(0X00020000)  2,(0X00040000)  0X0003,(IPL0)
2:      13,(0X001A0000)  1,(0X00020000)  2,(0X00040000)  0X0003,(IPL1)
3:      16,(0X00200000)  1,(0X00020000)  2,(0X00040000)  0X0001,(IPL_CUST0)
4:      19,(0X00260000)  1,(0X00020000)  2,(0X00040000)  0X0001,(IPL_CUST1)
5:      22,(0X002C0000)  2,(0X00040000)  4,(0X00080000)  0X0006,(UBOOT0)
6:      28,(0X00380000)  2,(0X00040000)  4,(0X00080000)  0X0006,(UBOOT1)
7:      34,(0X00440000)  1,(0X00020000)  1,(0X00020000)  0X000D,(ENV0)
8:      36,(0X00480000)  1,(0X00020000)  1,(0X00020000)  0X000D,(ENV1)
9:      38,(0X004C0000)  1,(0X00020000)  0,(0000000000)  0X0040,(KEY_CUST)
10:     39,(0X004E0000)  3,(0X00060000)  0,(0000000000)  0X0002,(LOGO)
11:     42,(0X00540000)  40,(0X00500000)  0,(0000000000)  0X0012,(KERNEL)
12:     82,(0X00A40000)  40,(0X00500000)  0,(0000000000)  0X0009,(RECOVERY)
13:    122,(0X00F40000) 902,(0X070C0000)  0,(0000000000)  0X0021,(UBI)
dd if=/dev/zero bs=2k count=2 | tr '\000' '\377' > /home/xinhua.ye/alkaid_mp/project/image/output/images/cis.bin
```

b. In the kernel stage, after the kernel is up, you can check it by typing the command: cat /proc/mtd

```
/ # cat /proc/mtd
dev:      size    erasesize  name
mtd0: 00060000 00020000 "IPL0"
mtd1: 00060000 00020000 "IPL1"
mtd2: 00060000 00020000 "IPL_CUST0"
mtd3: 00060000 00020000 "IPL_CUST1"
mtd4: 000c0000 00020000 "UBOOT0"
mtd5: 000c0000 00020000 "UBOOT1"
mtd6: 00040000 00020000 "ENV"
mtd7: 00040000 00020000 "ENV1"
mtd8: 00020000 00020000 "KEY_CUST"
mtd9: 00060000 00020000 "LOGO"
mtd10: 00500000 00020000 "KERNEL"
mtd11: 00500000 00020000 "RECOVERY"
mtd12: 070c0000 00020000 "UBI"
```

c. In the boot stage, you can type the command: mtdpart in the uboot console (Nor flash uses t.list)

```
SigmaStar # mtdparts
device nand0 <nand0>, # parts = 13
#: name          size          offset         mask_flags
0: IPL0          0x00060000    0x00140000     0
1: IPL1          0x00060000    0x001a0000     0
2: IPL_CUST0     0x00060000    0x00200000     0
3: IPL_CUST1     0x00060000    0x00260000     0
4: UBOOT0       0x000c0000    0x002c0000     0
5: UBOOT1       0x000c0000    0x00380000     0
6: ENV          0x00040000    0x00440000     0
7: ENV1         0x00040000    0x00480000     0
8: KEY_CUST     0x00020000    0x004c0000     0
9: LOGO         0x00060000    0x004e0000     0
10: KERNEL      0x00500000    0x00540000     0
11: RECOVERY    0x00500000    0x00a40000     0
12: UBI         0x070c0000    0x00f40000     0

active partition: nand0,0 - (IPL0) 0x00060000 @ 0x00140000

defaults:
mtdids : nand0=nand0
mtdparts: mtdparts=nand0:0xc0000@0x140000(NPT), -(UBI)
```

1.2. Modify the partition size

a. 找到自己项目使用的对应分区config文件。如下图，找到 IMAGE_CONFIG 变量指定的分区配置config，这里拿spinand.ubifs.p2.partition.config举例（目

录:project\image\configs\i2m\spinand.ubifs.p2.partition.config)

```
TOOLCHAIN_VERSION = 8.2.1
KERNEL_VERSION = 4.9.84
LIBC = libc-2.28
BUSYBOX = busybox-1.20.2-arm-linux-gnueabi-hf-glibc-8.2.1-dynamic
KERNEL_CONFIG = glibc
IMAGE_CONFIG = spinand.ubifs.p2.partition.config
CUSTOMER_OPTIONS = 011a.201_options.mk
CUSTOMER_TAILOR = nvr_i2m_display_glibc_tailor.mk
MMAP = MMAP_I2M_128M.h
MHAL = i2m
```

b. 打开分区配置config，这里拿kernel分区为例，如下图，修改*\$(PATSIZ) 制定的大小值即可

```
61 kernel$(RESOUCE) = $(PROJ_ROOT)/release/$(PRODUCT)/$(CHIP)/$(BOARD)/$(TOOLCHAIN)/$(TOOLCHAIN_VERSION)/bin/kernel/$(FLASH_TYPE)/uImage.kz
62 kernel$(PATSIZ) = 0x500000
63 kernel$(BOOTENV) = $(KERNEL_BOOT_ENV)
64 kernel$(MTDPART) = $(kernel$(PATSIZ))(KERNEL),$(kernel$(PATSIZ))(RECOVERY)
65 kernel$(OTBLK) = /dev/mtd10
66
```

注：因为flash的物理特性，一般情况下擦除只能按照一个block为单位，所以分区配置要按block大小对齐（nand flash 分区大小需按照128K对齐，nor flash按照64K对齐）

1.3. 配置修改Nand Flash的总大小

a. 修改文件：alkaid\project\image\image.mk

b. 找到cis_nofsimage的目标字段，将其中的-k 参数的数值改写，如下图-k 1024，代表的是 1024*128K = 128M

注：如果是使用的256M的flash，则 -k 1024需要改为 -k 2048（2048*128K = 256M）

```
cis_nofsimage:
@echo [[$@]]
cp -vf $(PROJ_ROOT)/board/$(CHIP)/boot/$(FLASH_TYPE)/partition/* $(IMAGEDIR)/boot/
$(PROJ_ROOT)/image/makefiletools/bin/pnigenerator -s 64 -p 2048 -b 64 -k 1024 -u 8 -l 0x20000 -t "$(cis$(PATSIZ))(CIS),$(
dd if=/dev/zero bs=$(cis$(PGSIZE)) count=2 | tr '\000' '\377' > $(cis$(RESOUCE))
dd if=$(IMAGEDIR)/boot/GCIS.bin of=$(cis$(RESOUCE)) bs=$(cis$(PGSIZE)) count=1 conv=notrunc seek=0
dd if=$(IMAGEDIR)/boot/PARTINFO.pni of=$(cis$(RESOUCE)) bs=$(cis$(PGSIZE)) count=1 conv=notrunc seek=1
cat $(IMAGEDIR)/boot/SPINANDINFO.sni >> $(cis$(RESOUCE))
```

2. DDR的内存查看与配置

2.1. 平台内存的组成方式

DDR总内存 = linux内存 + kernel reserved部分内存 (包含MMA)

2.2. 查看内存的方法

a. 查看linux内存：

为了准确的查看内存的MemFree，建议查看前先执行：`echo 3 >`

`/proc/sys/vm/drop_caches`，然后再`cat /proc/meminfo`，如下图，MemTotal代表的是linux的总内存大小，MemFree就是空闲内存

```
/ # echo 3 > /proc/sys/vm/drop_caches
sh (673): drop_caches: 3
/ # cat /proc/meminfo
MemTotal:        64004 kB
MemFree:         48700 kB
MemAvailable:    47968 kB
Buffers:         0 kB
Cached:          984 kB
SwapCached:      0 kB
Active:          1216 kB
Inactive:        40 kB
Active(anon):    292 kB
Inactive(anon):  0 kB
Active(file):    924 kB
Inactive(file):  40 kB
Unevictable:     0 kB
Mlocked:         0 kB
SwapTotal:       0 kB
SwapFree:        0 kB
Dirty:           0 kB
Writeback:       0 kB
AnonPages:       308 kB
Mapped:          932 kB
Shmem:           0 kB
Slab:            6068 kB
SReclaimable:    540 kB
SUnreclaim:      5528 kB
KernelStack:     568 kB
PageTables:      56 kB
NFS_Unstable:    0 kB
Bounce:          0 kB
WritebackTmp:    0 kB
CommitLimit:     32000 kB
Committed_AS:    1128 kB
VmallocTotal:    909312 kB
VmallocUsed:     0 kB
VmallocChunk:    0 kB
CmaTotal:        2048 kB
CmaFree:         1352 kB
/ #
```

b. 查看MMA内存：

如下图，length代表分配的MMA总大小为 0x2800000 Byte，其中MMA剩余的内存为：chunk_mgr_avail = 0x37df000 Byte

```

/ # cat /proc/mi_modules/mi_sys_mma/mma_heap_name0
      mma_heap_name      heap_base_cpu_bus_addr      length      chunk_mgr_avail
      mma_heap_name0      24400000      3800000      37df000
chunk_mgr info:
      offset      length      avail
      0      3800000      37df000
each chunk info:
      offset      length      used_flag      task_name      pid      Module
      0      20000      1      CMDMEM      -1      mi_sys
      20000      1000      1      sys-logConfig      -1      mi_sys
      21000      37df000      0      NA      -1      mi_sys
/ #

```

2.3. 配置MMA内存大小的方法

a. 项目工程中，打开自己项目project的config（这里使用：configs/nvr/i2m/8.2.1/spinand.glibc.011a.128）。如下图，找到文件中的CONFIG_KERNEL_BOOT_ENV,修改mma_heap_name0对应的sz（这里sz=0x3800000 即56M大小的MMA）

```

16 MERGE_BOOT = TRUE
17 BOOTLOGO_FILE = sigmastar1024_600.jpg
18 BOOTLOGO_ADDR = E_LX_LOGO_RESERVED_FB
19 DISP_OUT_NAME = SAT070CP50
20 EXBOOTARGS =
21 KERNEL_BOOT_ENV = LX_MEM=$(KERNEL_MEMLN) mma_heap=mma_heap_name0,miu=0,sz=0x3800000 mma_memblock_remove=1 highres=off
22 TOOLCHAIN_REL = arm-linux-gnueabihf-

```

b. 在debug过程中，可以在boot控制台直接修改板子上的bootargs参数，saveenv之后reset重启即可生效，如下图

```

SigmaStar # setenv bootargs 'console=ttyS0,115200 ubi.mtd=UBI,2048 root=ubi:rootfs ro rootfstype=ubifs init=/linux
,miu=0,sz=0x3800000 mma_memblock_remove=1 highres=off mmap_reserved=fb,miu=0,sz=0x300000,max_start_off=0x7C00000,m
84k(IPL1),384k(IPL_CUST0),384k(IPL_CUST1),768k(UBOOT0),768k(UBOOT1),256k(ENV),256k(ENV1),0x20000(KEY_CUST),0x60000
SigmaStar # saveenv

```

2.4. 如何判断MMA/Linux不足够

a. 如果log出现MmaAlloc fail等字样，说明MMA内存可能不够，要确认问题，可以在出现问题之前：echo 3 > /proc/mi_modules/mi_sys/debug_level 调高sys的debug level，出现问题的时候就会在串口打印出问题MMA debug信息

```

[MI ERR ]: MI_SYS_IMPL MmaAlloc[6195]: fail
[MI ERR ]: mi_sys_alloc_from_mma_allocators[940]: Alloc buf:linear-frm-buf in mma fail! size:0x2ee000
[MI ERR ]: MI_SYS_IMPL MmaAlloc[6195]: fail
[MI ERR ]: _MI_VDEC_IMPL AllocateDecFrameBuffer[1897]: Failed to allocate frame buffer
[MI ERR ]: _MI_VDEC_OnPollingAsyncOutputTaskConfig[5412]: Failed to allocate frame buffer for chn 0 Init
[MI ERR ]: _MI_VDEC_IMPL_CloseDecoderInternal[2833]: chn0 occur err due to shortage of memory, please stop/close this channel!

```

b. 如果是Linux内存不足，一般会出现OOM，OOM会kill掉某些应用，log产生backtrace信息

```

Mem-Info:
active_anon:3017 inactive_anon:0 isolated_anon:0
 active_file:15 inactive_file:29 isolated_file:0
 unevictable:0 dirty:0 writeback:0 unstable:0
 slab_reclaimable:147 slab_unreclaimable:1295
 mapped:2 shmem:0 pagetables:30 bounce:0
 free:153 free_pcp:0 free_cma:0
Node 0 active_anon:12068kB inactive_anon:0kB active_file:60kB inactive_file:116kB unevictable:0kB
 nstable:0kB pages_scanned:14 all_unreclaimable? yes
Normal free:612kB min:624kB low:780kB high:936kB active_anon:12068kB inactive_anon:0kB active_file
 slab_reclaimable:588kB slab_unreclaimable:5180kB kernel_stack:560kB pagetables:120kB bounce:0kB f
lowmem_reserve[]: 0 0
Normal: 19*4kB (UM) 5*8kB (UM) 3*16kB (UM) 0*32kB 1*64kB (M) 1*128kB (M) 1*256kB (M) 0*512kB 0*102
44 total pagecache pages
16352 pages RAM
0 pages HighMem/MovableOnly
9734 pages reserved
512 pages cma reserved
Out of memory: Kill process 782 (SwapTest) score 414 or sacrifice child
Killed process 782 (SwapTest) total-vm:12688kB, anon-rss:11264kB, file-rss:8kB, shmem-rss:0kB
Killed

```

2.5. 内存问题如何DEBUG

项目上，只有通过调节MMA的内存大小，来合理的均衡MMA跟linux的可用内存之间的大小，内存相关的问题可以按照以下步骤的顺序按步确认。

- 确定是否存在有泄漏问题？如果存在可用内存不断减少的情况，请先解决内存泄漏问题
- 是否存在Out of Memory的情况？如果出现这种情况，说明内存不足，如果MMA有剩余可以考虑将MMA的内存分配减小，相应的减小部分就会增加到linux内存部分
- 是否出现MmaAlloc fail相关的MMA申请错误字样？如果出现这种情况，说明该使用场景下，MMA的内存大小不足以支持，如果linux部分Memory有剩余，可以增大MMA部分内存
- 在没有出现内存泄漏，且linux内存跟MMA内存互相均衡的情况下，内存依旧不足够使用？如果出现这种情况，只能考虑做下功能裁剪或者更换内存更大的DDR